LMM LC2

https://gemini.google.com/app/0501596bd01d4da9

*User prompt: como evito o overflow de x * y? divido por y e vejo se da x? ou esse truque ja nao funcionara para a divisao?*

Response: O truque de dividir por y e verificar se dá x (ou seja, testar se `(x * y) / y == x`) **não é fiável nem seguro** para detetar ou prevenir *overflow* em vetores de bits (*BitVectors*) de tamanho fixo.

Por que razão este truque falha?

1. **A divisão por zero ($y = 0$):** Se o programa permitir que y seja zero naquele momento, a divisão torna-se matematicamente indefinida ou gera comportamentos anómalos no Z3, invalidando a verificação.
2. **Aritmética modular (2^n):** Como os *BitVectors* operam com tamanho fixo e dão "voltas" cíclicas módulo 2^n, o produto de uma multiplicação com *overflow* pode sofrer truncações. Devido a essas propriedades modulares, existem casos em que o valor truncado, ao ser dividido por y, coincide por mero acaso com o x original, fazendo com que o solver ignore um *overflow* real (gerando falsos negativos).
3. **Custo computacional para o Solver:** Os *solvers* SMT como o Z3 processam muito melhor operações puras de multiplicação e expansão de bits do que divisões em vetores de bits, que geram circuitos lógicos mais complexos de resolver (*bit-blasting*).

Como se deve fazer corretamente?

A abordagem padrão e rigorosa para detetar *overflow* numa multiplicação de *BitVectors* passa por uma de duas vias:

- **Extensão de Bits (*ZeroExt* ou *SignExt*):** Em vez de dividires o resultado final, duplicas preventivamente a capacidade dos operandos antes de multiplicar. Por exemplo, se estás a trabalhar com n bits, podes estender x e y para $2n$ bits usando `ZeroExt`, efetuar a multiplicação nesse novo espaço maior e verificar se a metade superior do resultado é estritamente zero. Como o espaço tem o dobro do tamanho, nunca haverá *overflow* oculto.
- **Validação prévia de limites:** Garantir através de operadores sem sinal (`ULT`, `UGT`) que o produto dos valores não ultrapassa o limite máximo representável antes de avançar com a atualização.

User prompt: ou seja se usar 8 bits nos vecs ele calcula em 16 bits e verifica se os 8 primeiros bits estao limpos?

Response: Exatamente! Compreendeste o conceito de forma perfeita.

Ao fazeres uma **extensão de bits** (como o `ZeroExt` no Z3), o que acontece nos bastidores é exatamente isso:

1. Pegas em dois valores de 8 bits e convertes-os temporariamente para 16 bits antes de os multiplicar.

2. Como o espaço de 16 bits é o dobro do tamanho, ele acomoda confortavelmente o resultado máximo possível da multiplicação sem sofrer truncações (já que 8 bits $\times$ 8 bits cabem sempre num resultado de até 16 bits).
3. Para confirmar se houve ou não *overflow* face aos limites originais, basta verificar se a metade superior do resultado de 16 bits (os 8 bits mais significativos) é estritamente igual a zero. Se estiver tudo limpo (a zeros), significa que a operação coube perfeitamente nos 8 bits originais e não houve perda de dados.

Esta abordagem é totalmente rigorosa para o Z3 e evita os problemas associados à divisão ou a heurísticas menos seguras.

User prompt: qual 'e o formato de ZeroExt? e ele identifica o tamanho dos vecs usados ou tem de ser passado manualmente? explica me como funciona e como 'e usado

Response: No Z3, a função `ZeroExt` serve para estender um bit-vector adicionando zeros à esquerda (nos bits mais significativos, ou *MSB*).

1. Formato e Sintaxe

O formato básico é:

$$\mathrm{ZeroExt}(i, \mathrm{expr})$$

- **`i`**: O número de bits que queres **acrescentar** (não o tamanho final total!).
- **`expr`**: O bit-vector original que pretendes estender.

2. Como funciona o tamanho? (Manual vs Automático)

- **O tamanho original é detetado automaticamente:** O Z3 sabe exatamente qual é o tamanho de `expr` consultando a sua estrutura interna. Não precisas de lhe dizer qual é o tamanho de partida.
- **O número de bits a adicionar é passado manualmente:** Tens de indicar explicitamente a `i` quantos bits queres somar.

Por exemplo, se tens uma variável `x` de **8 bits** e queres elevá-la a **16 bits** para fazer uma multiplicação segura sem *overflow*:

- O tamanho atual é 8.
- Precisas de adicionar mais 8 bits para chegar a 16.
- Logo, escreves: `ZeroExt(8, x)`. O resultado será um novo bit-vector de 16 bits, onde os 8 bits mais à esquerda serão preenchidos estritamente com zeros e os 8 à direita mantêm o valor original de `x`.

3. Exemplo Prático de Uso na Multiplicação

Se quiseres usá-lo para evitar o *overflow* numa multiplicação de dois números de 8 bits (`x` e `y`) guardando o resultado em 16 bits, faz-se assim:

```
from z3 import *

# Declarar variáveis de 8 bits
x = BitVec('x', 8)
y = BitVec('y', 8)

# Estender ambas para 16 bits usando ZeroExt (adicionando 8 bits a cada uma)
x_16 = ZeroExt(8, x)
y_16 = ZeroExt(8, y)

# Multiplicar em 16 bits (espaço seguro, sem perdas nem overflow modular)
resultado_16 = x_16 * y_16

# Para garantir que não houve overflow face aos 8 bits originais,
# verificamos se os 8 bits superiores do resultado são estritamente 0:
sem_overflow = Extract(15, 8, resultado_16) == 0
```

User prompt: Transforma o ex3.py num ficheiro Marimo (formato .py) mantendo estritamente inalterada toda a lógica e regras originais. Faz apenas as adaptações técnicas necessárias para a estrutura do Marimo (células, imports e etc...).

Response: I will now generate the Marimo notebook version of your integer multiplication script, wrapping the original logic into clean Marimo cells while preserving all functions and checks.

I have successfully structured your script into a fully compliant Marimo notebook script (`ex3_marimo.py`), separating the logic into neat reactive cells while keeping all your original logic, helper functions, and verification settings strictly identical.

User prompt: Da-me uma forma de obter o mesmo resultado do ZeroExt sem ter o mesmo custo computacional de um multiplicador de 256 bits. nao des o codigo, apenas a ideia para eu a executar

Response: Para evitar o custo computacional pesado de um multiplicador de $2n$ bits (como os 256 bits no teu caso de 128 bits), podes recorrer a uma de duas ideias alternativas que trabalham diretamente no espaço original de n bits:

1. A Abordagem por Limite e Divisão não-sinalizada (`UDiv`)

Em vez de multiplicares os números e verificares se houve perdas, podes **inverter a lógica matemática antes da multiplicação**:

- Sabes qual é o valor máximo absoluto que cabe em n bits sem sinal (ou seja, $2^n - 1$).
- Se y for diferente de zero, o produto $x \times y$ não rebentará o limite de n bits se e só se o operando x for menor ou igual ao resultado da divisão inteira entre o limite máximo e y.
- Podes expressar esta restrição utilizando a divisão não-sinalizada nativa do Z3 (`UDiv`) diretamente com variáveis de n bits. Desta forma, o solver valida a segurança através de uma simples comparação de limites e de uma divisão, evitando criar um circuito de multiplicação de dobro tamanho.

2. A Abordagem por Bits Significativos (Zeros à Esquerda)

Outra alternativa passa por analisar o espaço ocupado pelos operandos:

- O número de bits necessários para representar o produto de dois números é, no máximo, a soma dos bits efetivamente utilizados por cada um deles.
- Podes medir de forma lógica quantos bits à esquerda (mais significativos) estão a zeros em x e em y.
- Se a soma dos espaços livres de ambos for inferior ao limiar de segurança, a multiplicação geraria *overflow*. Podes usar restrições de contagem de zeros ou operadores bit a bit para barrar a transição antes de efetuar a operação matemática pesada.

User prompt: seria melhor usar bit vecs para o pc tambem? ou usar ints nao faz diferenca no custo computacional?

Response: Usar `Int` ou `BitVec` para o `pc` (Program Counter) tem impacto técnico, principalmente devido à **mistura de teorias** no Z3.

Aqui está uma análise de como isso afeta o desempenho e o comportamento do solver:

1. Mistura de Teorias (*Theory Mixing*)

No teu modelo atual, estás a usar `BitVec` para todas as variáveis de dados (`a`, `b`, `x`, `y`, `z`) e `Int` para o `pc`.

- O Z3 lida com `Int` usando a teoria de Aritmética Inteira (LIA).
- O Z3 lida com `BitVec` usando a teoria de Vetores de Bits (geralmente resolvida por *bit-blasting* para SAT). Quando misturas `Int` e `BitVec` no mesmo estado, o Z3 tem de gerir a comunicação entre duas teorias matemáticas distintas. Embora o Z3 faça isto muito bem, pode introduzir alguma sobrecarga (*overhead*) desnecessária.

2. Vantagens de usar `BitVec` também para o `pc`

Se transformares o `pc` num `BitVec` de tamanho pequeno (por exemplo, `n_bits` ou um tamanho mais restrito como 4 bits, já que os estados vão de 0 a 5):

- **Homogeneidade:** Todas as variáveis do sistema passam a pertencer à mesma teoria matemática (Vetores de Bits).
- **Eficiência do Solver:** Os solvers SMT adoram problemas puramente baseados em *BitVectors*, pois conseguem otimizar enormemente a conversão para circuitos booleanos (SAT).
- **Realismo de Hardware:** Na prática, um contador de programa (*program counter*) num processador real é um registo de bits, não um número inteiro infinito.

3. O impacto prático no custo computacional

Para um intervalo tão pequeno de estados (0 a 5), a diferença de desempenho bruto entre usar `Int` ou um `BitVec` pequeno para o `pc` **costuma ser marginal** porque o Z3 resolve restrições de igualdade simples (`pc == 0`, `pc != 4`) de forma quase instantânea em ambos os casos.

No entanto, **é preferível usar `BitVec` para o `pc`** por questões de consistência arquitetural e para evitar a mistura de teorias no Z3. Se decidires mudar, bastaria definir o `pc` como um `BitVec` (por exemplo, com o mesmo `n_bits` ou um tamanho fixo menor) e adaptar as comparações para valores literais de BitVec (ex: `BitVecVal(0, n_bits)`).

User prompt: o que faz o simplify e pode ser util? ou o ganho e' irrisorio

Response: A função `simplify` do Z3 serve para **simplificar algebraicamente e logicamente expressões**, aplicando reduções automáticas como:

- *Constant folding* (por exemplo, calcular automaticamente contas fixas como `2 + 3` para `5`).
- Simplificações lógicas e booleanas (por exemplo, reduzir `And(True, x)` para apenas `x`).
- Compactação de expressões redundantes.

É útil? O ganho de desempenho é irrisório?

A resposta divide-se em duas frentes: **desempenho computacional** vs. **legibilidade para humanos**.

1. Para o desempenho do Solver (Ganho Irrisório / Redundante)

No que toca à velocidade e eficiência do solver, o ganho de chamar o `simplify` manualmente no Python é **praticamente irrisório**. Os *solvers* SMT modernos (como o Z3) já possuem motores internos de pré-processamento e simplificação extremamente potentes (*preprocessing/simplification tactics*). Quando passas uma fórmula para um `Solver()`, ele próprio já a limpa e otimiza de forma muito mais agressiva do que a função `simplify` consegue fazer à superfície. Portanto, usá-la antes de adicionar restrições ao solver não acelera a execução e pode até gastar alguns ciclos de CPU desnecessariamente no lado do Python.

2. Para a depuração e leitura (Extremamente Útil)

A verdadeira utilidade do `simplify` brilha na **análise humana e no *debugging***. Quando estás a testar o código, a imprimir contraexemplos complexos (`solver.model()`) ou a inspecionar fórmulas longas geradas por ciclos, o resultado costuma ser uma árvore de expressões gigantesca e ilegível. Aplicar o `simplify` a esses resultados limpa a expressão, tornando-a muito mais compacta e fácil de ler para perceberes o que se está a passar no modelo.

Resumo: Não precisas (nem ganhas velocidade) de usar o `simplify` para alimentar o solver ou nas tuas funções lógicas de transição. Guarda-o estritamente para quando precisares de depurar e imprimir contraexemplos limpos na consola para análise.

User prompt: na funcao trans o que posso fazer para nao ter prox['a'] == curr['a'], prox['b'] == curr['b'] em todas as transicoes? ou tenho de por as igualdades em todas as transicoes?

Response: Para responder diretamente: **logicamente, sim, tens de garantir que essas igualdades existem** para todas as variáveis que não mudam. Se omitires a igualdade de uma variável no passo seguinte (`prox`), para o solver SMT essa variável passa a ser livre (não constrained), o que significa que ela pode assumir qualquer valor arbitrário de "lixo" na transição seguinte, corrompendo o

comportamento do autómato.

No entanto, **não precisas de escrever manualmente** `prox['a'] == curr['a']` em todas as alíneas e ramos de código repetidamente. Podes usar truques de programação em Python para automatizar essa imposição (a chamada *Frame Rule* na verificação formal).

Aqui tens as formas mais comuns de evitar essa repetição sem alterar a lógica:

1. Criar uma Função Auxiliar de "Frame" (Inercia)

Em vez de repetires as igualdades em cada And, podes definir uma função que diz quais são as variáveis que se mantêm inalteradas (por omissão) e aplicá-la onde precisar de persistência:

```
def unchanged(curr, prox, vars_list):
    # Gera automaticamente a igualdade para todas as variáveis passadas na lista
    return And([prox[v] == curr[v] for v in vars_list])
```

Assim, dentro de cada transição, em vez de listares uma a uma, podes apenas chamar:

```
unchanged(curr, prox, ['a', 'b', 'x', 'y', 'z'])
```

E depois sobrescreves apenas a variável que efetivamente mudou naquele passo específico (ex: `prox['x'] == curr['x'] << 1`).

2. Definir uma Lista Global de Variáveis de Dados

Podes listar todas as variáveis do modelo numa estrutura (por exemplo, uma lista) e estruturar a relação de transição de forma modular, indicando explicitamente apenas o que **muda** em cada instrução do autómato, deixando o resto sujeito à regra de inércia automática.

User prompt: explica-me o porque de def _(And, BitVecVal, LShR, Or, UGE, ULT, n_bits, simplify): def unchanged(curr, prox, vars_list): return And([prox[v] == curr[v] for v in vars_list]) def trans(curr, prox): return simplify(And(unchanged(curr, prox, ['a', 'b']), Or(And(# init (0) -> skip (1) curr['pc'] == BitVecVal(0, n_bits), prox['pc'] == BitVecVal(1, n_bits), prox['x'] == curr['a'], prox['y'] == curr['b'], prox['z'] == BitVecVal(0, n_bits)), And(# skip (1) -> left (2) curr['pc'] == BitVecVal(1, n_bits), curr['y'] != BitVecVal(0, n_bits), (curr['y'] & 1) == 0, prox['pc'] == BitVecVal(2, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# skip (1) -> right (3) curr['pc'] == BitVecVal(1, n_bits), curr['y'] != BitVecVal(0, n_bits), (curr['y'] & 1) == 1, prox['pc'] == BitVecVal(3, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# skip (1) -> stop (5) curr['pc'] == BitVecVal(1, n_bits), curr['y'] == BitVecVal(0, n_bits), prox['pc'] == BitVecVal(5, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# left (2) -> skip (1) curr['pc'] == BitVecVal(2, n_bits), UGE(curr['x'] + curr['x'], curr['x']), prox['pc'] == BitVecVal(1, n_bits), prox['x'] == curr['x'] << 1, prox['y'] == LShR(curr['y'], 1), unchanged(curr, prox, ['z'])), And(# left (2) -> error (4) curr['pc'] == BitVecVal(2, n_bits), ULT(curr['x'] + curr['x'], curr['x']), prox['pc'] == BitVecVal(4, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# right (3) -> skip (1) curr['pc'] == BitVecVal(3, n_bits), UGE(curr['z'] + curr['x'], curr['z']), prox['pc'] == BitVecVal(1, n_bits), prox['y'] == curr['y'] - 1, prox['z'] == curr['z'] + curr['x'], unchanged(curr, prox, ['x'])), And(# right (3) -> error (4) curr['pc'] == BitVecVal(3, n_bits), ULT(curr['z'] + curr['x'], curr['z']), prox['pc'] == BitVecVal(4, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# error (4) -> error (4) curr['pc'] == BitVecVal(4, n_bits), prox['pc'] == BitVecVal(4, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# stop (5) -> stop (5) curr['pc'] == BitVecVal(5, n_bits), prox['pc'] == BitVecVal(5, n_bits), unchanged(curr, prox, ['x', 'y', 'z']))))) return (trans,) ter poupado imenso tempo? 'e por apenas calcular prox['a'] == curr['a'] e prox['b'] == curr['b'] serem apenas calculados 1 vezes no inicio do trans com munchanged? e as restantes vezes que uso unchanged e' calculado como antes por ainda estarem no Or ou sao apenas calculas menos operacoes?

Response:

User prompt: explica-me o porque de def _(And, BitVecVal, LShR, Or, UGE, ULT, n_bits, simplify): def unchanged(curr, prox, vars_list): return And([prox[v] == curr[v] for v in vars_list]) def trans(curr, prox): return simplify(And(unchanged(curr, prox, ['a', 'b']), Or(And(# init (0) -> skip (1) curr['pc'] == BitVecVal(0, n_bits), prox['pc'] == BitVecVal(1, n_bits), prox['x'] == curr['a'], prox['y'] == curr['b'], prox['z'] == BitVecVal(0, n_bits)), And(# skip (1) -> left (2) curr['pc'] == BitVecVal(1, n_bits), curr['y'] != BitVecVal(0, n_bits), (curr['y'] & 1) == 0, prox['pc'] == BitVecVal(2, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# skip (1) -> right (3) curr['pc'] == BitVecVal(1, n_bits), curr['y'] != BitVecVal(0, n_bits), (curr['y'] & 1) == 1, prox['pc'] == BitVecVal(3, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# skip (1) -> stop (5) curr['pc'] == BitVecVal(1, n_bits), curr['y'] == BitVecVal(0, n_bits), prox['pc'] == BitVecVal(5, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# left (2) -> skip (1) curr['pc'] == BitVecVal(2, n_bits), UGE(curr['x'] + curr['x'], curr['x']), prox['pc'] == BitVecVal(1, n_bits), prox['x'] == curr['x'] << 1, prox['y'] == LShR(curr['y'], 1), unchanged(curr, prox, ['z'])), And(# left (2) -> error (4) curr['pc'] == BitVecVal(2, n_bits), ULT(curr['x'] + curr['x'], curr['x']), prox['pc'] == BitVecVal(4, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# right (3) -> skip (1) curr['pc'] == BitVecVal(3, n_bits), UGE(curr['z'] + curr['x'], curr['z']), prox['pc'] == BitVecVal(1, n_bits), prox['y'] == curr['y'] - 1, prox['z'] == curr['z'] + curr['x'], unchanged(curr, prox, ['x'])), And(# right (3) -> error (4) curr['pc'] == BitVecVal(3, n_bits), ULT(curr['z'] + curr['x'], curr['z']), prox['pc'] == BitVecVal(4, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# error (4) -> error (4) curr['pc'] == BitVecVal(4, n_bits), prox['pc'] == BitVecVal(4, n_bits), unchanged(curr, prox, ['x', 'y', 'z'])), And(# stop (5) -> stop (5) curr['pc'] == BitVecVal(5, n_bits), prox['pc'] == BitVecVal(5, n_bits), unchanged(curr, prox, ['x', 'y', 'z']))))) return (trans,) ter poupado imenso tempo? 'e por apenas calcular prox['a'] == curr['a'] e prox['b'] == curr['b'] serem apenas calculados 1 vezes no inicio do trans com munchanged? e as restantes vezes que uso unchanged e' calculado como antes por ainda estarem no Or ou sao apenas calculas menos operacoes?

Response: O facto de teres retirado o `unchanged(curr, prox, ['a', 'b'])` para fora do bloco `Or` foi exatamente o que te poupou um tempo de execução drástico. A tua intuição está certíssima, e o ganho ocorre em dois níveis distintos: na construção da fórmula em Python e na forma como o Z3 resolve o problema lógico.

1. A Lei Distributiva na Lógica Booleana

O que fizeste foi aplicar a propriedade distributiva da matemática à lógica booleana. No teu código antigo, a estrutura era: `(A e Ramo1) OU (A e Ramo2) OU ... OU (A e Ramo10)` *(onde A é a regra de que 'a' e 'b' não mudam).*

Ao puxares a inércia de 'a' e 'b' para fora, a fórmula passou a ser: `A E (Ramo1 OU Ramo2 OU ... OU Ramo10)`

Em vez de o Python gerar 20 equações de igualdade e o Z3 ter de as ler em 10 caminhos diferentes, a restrição foi declarada apenas uma única vez para toda a transição. A árvore de expressões (AST) que o Python envia para o Z3 ficou substancialmente mais pequena e limpa.

2. O Impacto no Motor do Solver (Z3)

Para o Z3, esta mudança arquitetural muda completamente a ordem pela qual ele avalia as possibilidades:

- **No método antigo:** O solver olhava para o `Or` e tinha de "ramificar" o seu pensamento em 10 caminhos possíveis. Só depois de entrar num ramo é que ele descobria a regra "ah, afinal o *a* e o *b* ficam iguais aqui". Ele repetia esta dedução 10 vezes.

- **No método novo:** Antes sequer de olhar para os 10 ramos do `Or`, o solver processa o And exterior. Ele "tranca" imediatamente os valores de `a` e `b` do estado anterior para o estado seguinte. Quando vai avaliar os caminhos do `Or`, o espaço de procura já está drasticamente reduzido porque o destino de `a` e `b` já é um facto absoluto, independentemente do ramo escolhido. Isto corta milhares de cálculos internos de propagação.

3. Sobre os `unchanged` dentro do `Or`

Respondendo à tua segunda pergunta: sim, os `unchanged(curr, prox, ['x', 'y', 'z'])` que ficaram dentro do `Or` continuam a ser calculados de forma condicional, exatamente como antes. O Z3 só os aplica se esse ramo específico for escolhido.

A diferença é que, como abstraíste as variáveis `a` e `b`, o peso global de cada ramo diminuiu. Menos operações lógicas por ramo significam que a função `simplify` que colocaste a envolver o `trans` tem muito menos trabalho a tentar otimizar a fórmula final, resultando numa execução global muito mais rápida e elegante.
